«Sovereign AI, Public Footprint - What a Press Release Reveals About the Royal Navy's Flagship »

This article is entirely based on a combination of Open Source research (OSINT), deduction, and AI inference.

RANDOMITY

JUN 12, 2026

SAGA is an AI-powered application that was deployed on the Royal Navy's flagship - HMS Prince of Wales, during Operation HIGHMAST, an eight-month mission in 2025 spanning the Mediterranean, the Middle

East and the Indo-Pacific.[1] SAGA “capture[s] lessons, review[s] mission data, and provid[es] tailored AI support in one environment.”

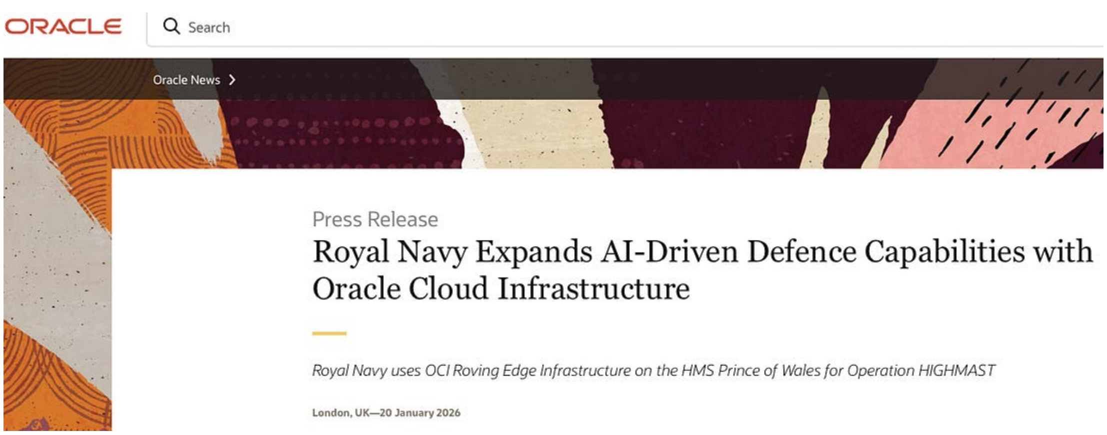
However, what makes SAGA particularly interesting as an AI cyber security study-case is that it is being deployed at a time where AI-security awareness is still low. A few years ago, Oracle’s public announcement that HMS Prince of Wales had deployed SAGA on OCI Roving Edge Infrastructure might have had no consequence. At the age of AI, a combination of OSINT[2], deduction, and AI inference[3] allows retro-engineering SAGA’s architecture on HMS POW.

This comes with obvious risks in terms of cybersecurity. This AI retro-engineering is all the more concerning as SAGA AI runs on the HMS Prince of Wales (HMS POW), one of the most critical assets of the RN. Due to its criticality the HMS POW should probably receive investment in priority for AI-powered defense against future drone swarms. An attacker’s retro-engineering of this AI in charge of the HMS POW’s anti-drone defense could platform on prior knowledge about SAGA’s potential similarities with it (e.g such as architectural similarities, use of similar civilian stack, potentially shared hardware). Such an attack would be even riskier than that of SAGA’s for the ship’s security, as well as a strategical risk for the RN.
Moreover, jeopardising the confidentiality of the HMS POW’s anti-drone AI would also be a financial risk for the RN, not only due to AI development but also as the RN’s aircraft carriers cannibalise the funding of the rest of the RN, whose finances are already running thin.[4]

Table of contents:

allowed to retro-ingineer SAGA’s likely architecture as deployed on the HMS Prince of Wales

3. Explanation of the final architecture obtained through reverse engineering
4. Architecture retro-engineering, would allow for attack preparation: a short risk analysis



1. The warning signs that AI-security awareness is still low are in public communication itself

Oracle publicly announced that HMS Prince of Wales had deployed SAGA AI on OCI[5] Roving Edge Infrastructure.[6] SAGA’s technical stack[7] is therefore partially identifiable through OSINT, and accelerated by AI-assisted analysis: an Oracle OCI, Whitespace/Collective application layer, that is deployed on ruggedized GPU hardware in DDIL[8] environments. The deployment was announced on an Oracle blog just weeks after Whitespace joined the Oracle Defense Ecosystem in June 2025.[9]
This exercise raises concerns about :
- how AI increases the need for a “culture of discretion”, including in political displays of military performance

- how important it is to invest in non-civilian solutions and prefer sovereign technologies to avoid such predictability. An analysis of the security risk benefit/financial cost balance should be performed prior to defaulting to civilian turnkey solutions.

2. A combination of OSINT research on Oracle’s website, AI architecture knowledge, and experience with industrial systems allowed to retro-engineer SAGA’s likely architecture

The results are as follows:

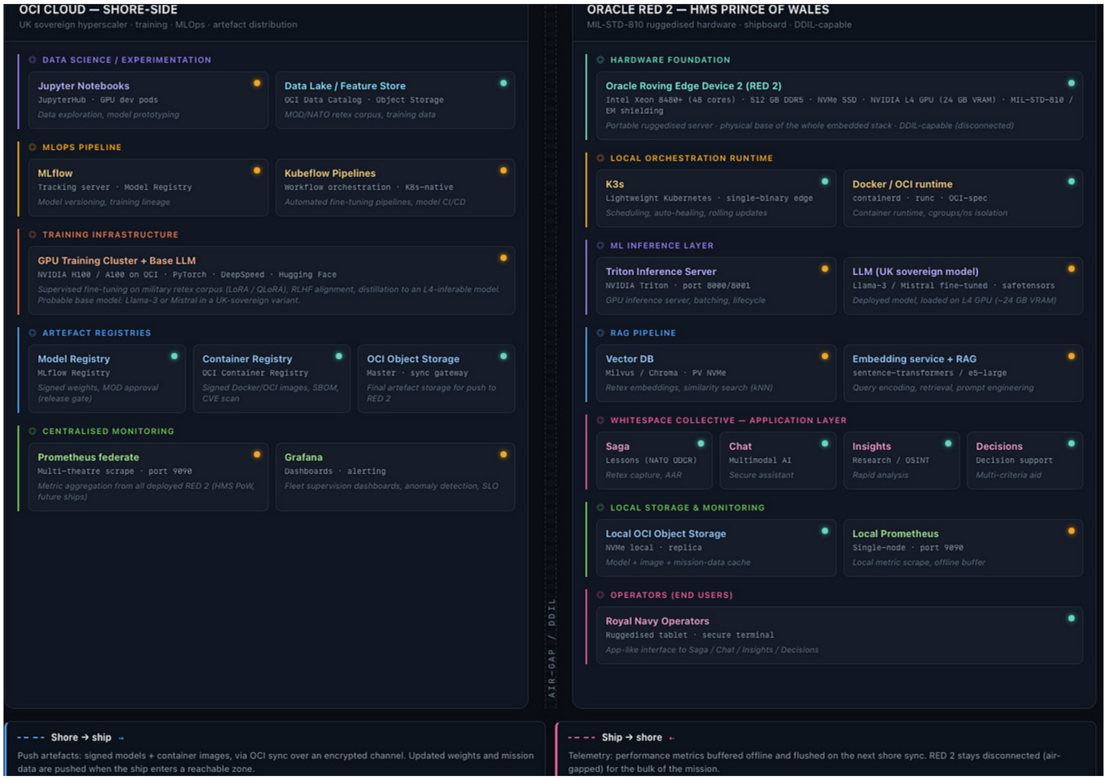
Component	Status	Rationale	Source
K3s / Kubernetes	Confirmed	Explicitly referenced in Oracle documentation for Roving Edge	Oracle Roving Edge Infrastructure docs (oracle.com/cloud/roving-edge-infrastructure/)
Docker / OCI containers	Confirmed	Foundation of entire OCI architecture	Oracle Roving Edge Infrastructure documentation + Oracle Cloud Infrastructure Blog, "Whitespace Brings Sovereign AI to the Tactical Edge with OCI" (January 2026) URL: https://blogs.oracle.com/cloud-infrastructure/whitespace-oci-sovereign-ai-tactical-edge https://www.oracle.com/cloud/roving-edge-infrastructure/
NVIDIA L4 GPU → Triton	Likely	L4 GPU = inference-optimized, standard NVIDIA stack for LLM deployment	Oracle Roving Edge Device 2 (RED 2) technical specifications: "Intel Xeon 8480+ (48 cores), 512 GB DDR5, NVMe SSD, NVIDIA L4 GPU (24 GB VRAM) optional" URL: https://www.oracle.com/cloud/roving-edge-infrastructure/ |
RAG pipeline	Highly likely	Saga = knowledge retrieval on lessons learned corpus, RAG is standard solution	Whitespace Collective website, Saga product description: "aligned with NATO's ODCR framework (Observations, Discussions, Conclusions, Recommendations)" and "capture, structuring and reuse of lessons learned" URL: https://white.space/collective-os-for-defence |

<table>
<tr>
<td>Triton Inference Server</td>
<td>| Likely |</td>
<td>- L4 GPU is inference-specific — not a training GPU (that would be A100/H100). Its presence strongly implies a production inference server.
- Triton's dynamic batching is critical for DDIL environments where multiple operators might query Saga simultaneously but intermittently — it optimizes GPU utilization.
- Kubernetes-native deployment — Triton deploys as a container managed by K3s, which we know is on the RED 2.
- Standard practice — In 2026, when deploying LLMs on NVIDIA GPUs in production, Triton is the reference implementation unless there's a specific reason to deviate.
- Alternatives: TorchServe, TensorFlow Serving, or proprietary Whitespace solution, but Triton represents best architectural fit given confirmed components (L4 GPU + K3s + LLM workload)</td>
<td>Inference from NVIDIA L4 GPU presence (L4 is inference-optimized) + standard NVIDIA deployment stack for LLM serving | | Vector DB | Likely | Inference from RAG architecture requirements (not explicitly documented) |</td>
</tr>
</table>

		workload).	
Vector DB (Chroma, Milvus...)	Likely	Required for RAG, but specific implementation unknown	Inference from RAG requirements
MLflow / Kubeflow	Unknown	Useful for shore-side experiment tracking, but Whitespace may have proprietary tooling	Not documented publicly
Prometheus / Grafana	Unknown	OCI native monitoring exists, but specific stack not documented	Not documented publicly
Jupyter	Unlikely in prod	Development environment, not deployed on classified embedded system	Standard ML production practice (dev environment, not production deployment)
Which LLM?	Classified	OCI provides access to Llama, Cohere, etc. — but UK sovereign model not disclosed	Not disclosed in public Oracle or Whitespace documentation |

By combining open-source confirmed architectural features, and inferring the remaining components from standard MLOps deployment patterns[10], one obtains two levels of architectures. The first one describes OCI Cloud: a cloud land-based infrastructure that SAGA connects to for model updates and receiving mission data). The second one is RED 2: the shipboard hardware SAGA relies on.
The first diagram shows the data flows between OCI Cloud and RED2, and the second one focuses on mapping RED2's internal architecture (from user inputing Saga to Saga's output).

Diagram N°1: OCI Cloud (shore) <-> RED 2 (ship) simplified diagram



This diagram illustrates a generic ML pipeline. In the case of HMS Prince of Wales, the Data Collection and Model Training phases (phase 1 to phase 3) are simplified: the base model (Llama-3 or Mistral) is acquired pre-trained, and only supervised fine-tuning on a military corpus is carried out onshore. However, this represents the entire hidden supply-chain, and constitutes the real attack surface an attacker would most likely consider first in terms of low detectability and impact on the LLM model. Keeping this in mind, let's proceed with understanding the HMS Prince of Wales AI architectural context (phase 3 to 5 of this diagram).

As seen previously, the OCI Coud (on-shore infrastructure) is in charge of:

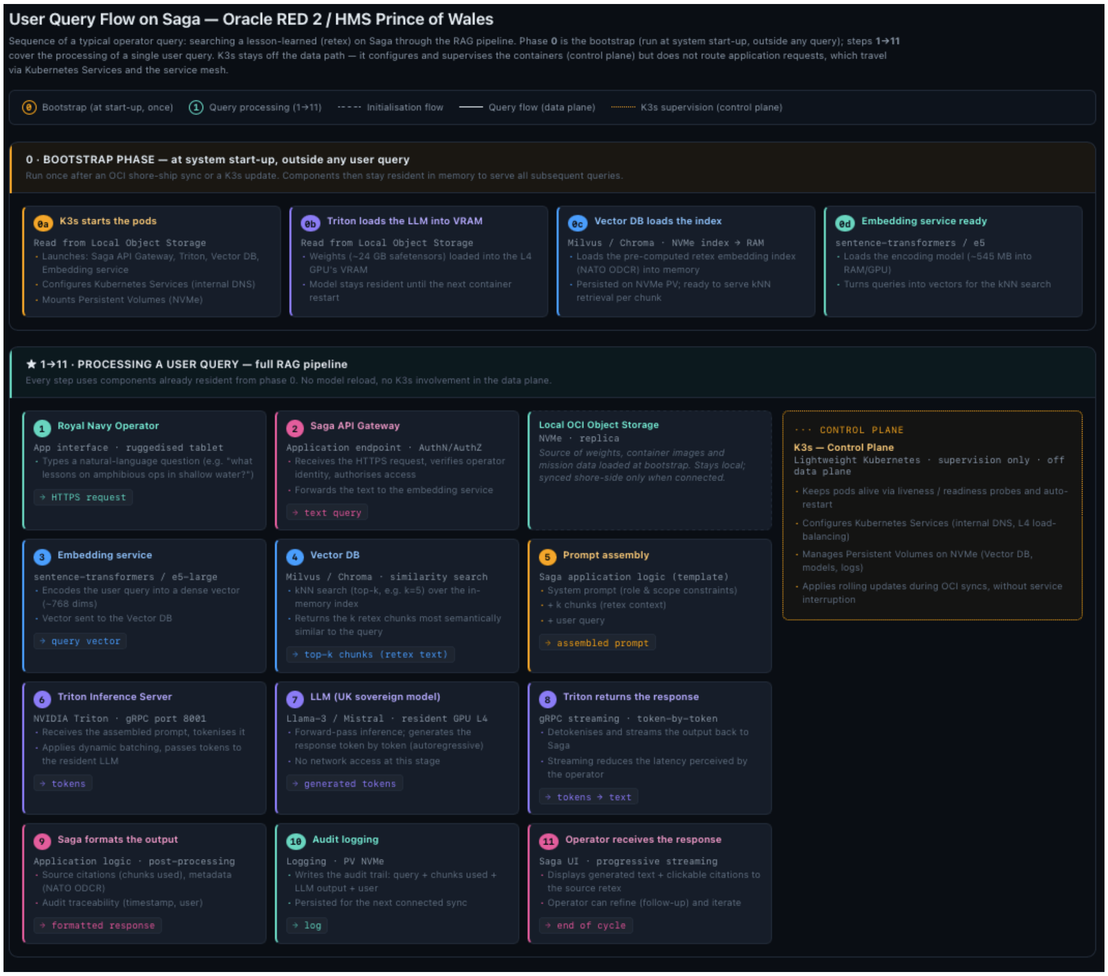
- Fine-tuning the acquired pre-trained Base LLM[11] model on military corpus[12] using **Kubeflow**. The final result is a fine-tuned LLM, called an "artefact") that retains general language understanding from Llama-3To, and specializes in naval military domain(terminology, doctrine, tactical scenarios). This combination allows it to answer queries like “What retex do we have on mine countermeasures in shallow water?” with relevant NATO ODCR-formatted responses. To do so, Kubeflow orchestrates the following tasks :

    - It launches the fine-tuning of the pre-trained LLM models using LoRA/QLoRA (and of older artefact versions in the future)
    - It orchestrates the different steps constituting the training sequence of an LLM (test → validation → packaging → push vers registry) and pushes it to the model registry.
    - Converting artefacts into “lighter and simplified” ship-board versions using pruning and quantitisation methods (the packaging phase).

- Tracking model training history using the MLFlow tracking server. This consists in :
    - Versioning the successive artefacts (weights V1, V2, V3, etc.) stored in the Model Registry (called “Object Storage”). It also tracks lineage (which models has been fine-tuned from which previous model).
- Synchronising RED2’s deployed shipboard LLM when the ship’s is in a reaching zone, which involves OCI:
    - pushing signed updated artefacts (updated weights) trough encrypted communication, as well as mission data
    - collecting performance metrics from RED2 (done by Prometheus)

After having described how RED2 functioning relies on OCI Cloud, the next step requires to dive into RED2’s internal architectural flow.

Diagram N°2: User query flow (simplified)



3. Explanation of the final architecture obtained through reverse engineering

To make the final result of this retro-engineering attempt of SAGA understandable to a non-techncal audiance, an explanataion of its key components and their functions is necessary(K3s, Triton, Vector DB, LLM)[13]

1. When a user sends a request to Saga from a tablet, the https request is sent to Saga's API gateway, which authentifies the user

and transfers the text request to the embedding server (this is called “activating the RAG pipeline”).

2. The embedding server converts the text into a single vector with a unique purpose: finding the most similar vectors in the Vector Database (where each document of the military corpus, including mission data, is converted into vectors).
3. The Vector Database powers a similarity search, finds the most similar vectors, and then proceeds to kNN-search outputting the k-number of chunks of texts that are the most similar to the user’s request.
4. SAGA then assembles the prompt with the following elements
    a. The System Prompt : these are admin content restrictions restricting the SAGA’s answers to only provide answer sticking to its defined role (it would probably prevent SAGA from providing out-of-scope financial advice for exemple).
    b. The k-chunks of text
    c. The user’s initial text request
    SAGA sends the assembled prompt to the Triton inference server
5. The Triton inference then performs 3 critical tasks:
    a. Spliting the assembled prompt into an LLM-friendly format called tokens.
    b. Dynamic batching
    Triton waits to receive a given amount of nealry concomittent requests before sending it to the LLM, thus making GPU-consumption economies of scale. This is critical as it is most likely officers would send requests at the samez time during a briefing or during ongoing operations.
    c. Model Lifecycle Management
    1. When the LLM is first-started, Triton retrieves the weights from the Local Object Storage (NVMe) and loads them on L4

GPU’s memory (named VRAM)
2. Triton then launches PyTorch code guides the LLM in the calculation steps it should perform and with which weights, to process the assembled prompt it received from SAGA.
3. Once these weights are in VRAM, the GPU can execute the model architecture (defined in the Triton/PyTorch code) to perform inference. The weights remain in VRAM until the next time the Triton container is restarted.

6. The LLM loaded in the VRAM processes the assembled prompt following PyTorch’s instructions and produces an inference.
    a. Triton then:
        i. translates the LLM’s answer from token back into text
        ii. Sends it back to SAGA’s API gateway
7. Saga then converts the answer into a user-friendly format, and stores a log if the audit trail (a trace of the original request, which k-chunks were used to answer it, and what the LLM output was).
8. The user receive the answer with the corpus of text that were mobilised to obtain it.
9. K3s orchestrates the deployment of this 8-step sequence by:
    a. Starting the pods (Saga API Gateway, Triton, Vector DB, Embedding service, Collective Apps) and configuring Kubernetes Services (internal DNS)
    b. Maintaining pod health through liveness/readiness probes and auto-restart (necessary when a new model needs to be loaded on the Triton Inference server)
    c. Configuring Kubernetes Services for internal DNS and L4 load-balancing
    d. Managing Persistent Volumes on NVMe for Vector DB, models, and logs;

e. And applying rolling updates during OCI sync without service interruption.

4. Architecture retro-engineering, would allow for attack preparation: a short risk analysis

Oracle's publication of the RN's use of OCI Roving Edge infrastructure during operation Highmast provided technical details that, when combined with the strong probability of the Royal Navy to build its AIs upon civilian LLMs[14] allowed to draw a rather likely picture of SAGA's architecture.
The real danger lies in the fact reconnaissance (including mapping) is an attacker's first step...
AI demultiplies an attacker's initial skills, while publicly documented stacks lower attackers's reconnaissance cost to near-zero, making it likelier to materialise.
Therefore, more than ever, information disclosure regarding projects involving national security should be be stricter than ever, even if it comes at the cost of forbidding political communication about defense investment. This posture would be coherent with the posture of several firms moving from open-source software distribution to proprietary-licence models: in April 2026, Cal.com, one of the world's largest open-source startups, announced it was going closed source, citing AI-driven vulnerability scans as the primary reason.[15] Would the British Government be capable of extending or imposing its military's culture of silence to its private industrial contractors ?

This posture would require extending the British military's culture of operational discretion to the public communications of private industrial contractors, a dimension that existing defence procurement security frameworks were not designed to govern.

Thanks for reading! Subscribe for free to receive new posts and support my work.

Subscribed

Footnotes

1 Trevithick, Joseph. ‘Royal Navy May Sacrifice Its Last Amphibious Ships to Pay For Its New Carriers’. _The War Zone_, 18 October 2017. https://www.twz.com/15279/royal-navy-may-sacrifice-its-last-amphibious-ships-to-pay-for-its-new-carriers.

2 OSINT: Open-source Intelligence

3 AI inference: an AI’s processing of the input it has been provided, resulting in an output

4 ‘Royal Navy Expands AI-Driven Defence Capabilities with Oracle Cloud Infrastructure’. 12 June 2026. https://www.oracle.com/uk/news/announcement/royal-navy-expands-ai-driven-defence-oci-2026-01-20/.

5 OCI: Oracle Cloud Infrastructure

6 ‘Royal Navy Expands AI-Driven Defence Capabilities with Oracle Cloud Infrastructure’. 12 June 2026. https://www.oracle.com/uk/news/announcement/royal-navy-expands-ai-driven-defence-oci-2026-01-20/.

7 Sets of technologies that are stacked together to build any application.

8 DDIL is an acronym standing for Denied, Degraded, Intermittent, and Limited (or Sometimes Disrupted and Limited). It refers to IT or communications environments experiencing an unstable, limited or unavailable network.

9 Crooks, Kelly. _Whitespace and Oracle Cloud Infrastructure: Accelerating Sovereign AI for Defense at the Tactical Edge_. 12 June 2026. https://blogs.oracle.com/cloud-infrastructure/whitespace-oci-sovereign-ai-tactical-edge.

10 Yet again, the architectural choices are made bearing in mind that the cost of sovereign LLM data gathering and training does not match with the RN's current financial situation.

11 such as Llama-3-70B or Mistral-Large

12 Such as NATO ODCR Lessons Learned or MOD Classified Retex, etc.

13 TryHackMe - AI Security Path

14 The cost of LLM data gathering and training does not match with the RN's current financial situation

15 'Cal.com is switching to a closed-source model: why AI security necessitates this decision | Cal.com - Online booking scheduling software'. 14 April 2026. [https://cal.com/fr/blog/cal-com-goes-closed-source-why]

Bibliography:

---

1. Trevithick, Joseph. 'Royal Navy May Sacrifice Its Last Amphibious Ships to Pay For Its New Carriers'. _The War Zone_, 18 October 2017. [https://www.twz.com/15279/royal-navy-may-sacrifice-its-last-amphibious-ships-to-pay-for-its-new-carriers]

(https://www.twz.com/15279/royal-navy-may-sacrifice-its-last-amphibious-ships-to-pay-for-its-new-carriers).

2. ‘Royal Navy Expands AI-Driven Defence Capabilities with Oracle Cloud Infrastructure’. 12 June 2026. https://www.oracle.com/uk/news/announcement/royal-navy-expands-ai-driven-defence-oci-2026-01-20/.
3. ‘Royal Navy Expands AI-Driven Defence Capabilities with Oracle Cloud Infrastructure’. 12 June 2026. https://www.oracle.com/uk/news/announcement/royal-navy-expands-ai-driven-defence-oci-2026-01-20/.
4. Crooks, Kelly. _Whitespace and Oracle Cloud Infrastructure: Accelerating Sovereign AI for Defense at the Tactical Edge_. 12 June 2026. https://blogs.oracle.com/cloud-infrastructure/whitespace-oci-sovereign-ai-tactical-edge.
5. Crooks, Kelly. _Whitespace and Oracle Cloud Infrastructure: Accelerating Sovereign AI for Defense at the Tactical Edge_. 12 June 2026. https://blogs.oracle.com/cloud-infrastructure/whitespace-oci-sovereign-ai-tactical-edge.
6. TryHackMe - AI Security Path
7. ‘Cal.com is switching to a closed-source model: why AI security necessitates this decision | Cal.com - Online booking scheduling software’. 14 April 2026. [https://cal.com/fr/blog/cal-com-goes-closed-source-why]

Written by Gaudet P.